Universidade da Beira Interior
Departamento de Informática

Nº 50391 - 2026 *Testing System for Java Payara APIs: An Automated Regression and CI/CD Framework for the GNSS EPOS API*

Author:

Eduardo Pires Valentim de Almeida Abrantes

Supervisor and Co-Supervisor:

Professor Paul Andrew Crocker
Vasco Senra

July 5, 2026

Acknowledgments

The completion of this project, and indeed of a large part of my academic journey, would not have been possible without the help, guidance and patience of many people. I would first like to thank my supervisor, Professor Paul Andrew Crocker, and my co-supervisor, Vasco Senra, for proposing this project and for the technical direction, the critical reviews and the trust placed in me throughout the development of the work described in this report.

A special word of gratitude goes to the staff of the SEGAL Laboratory at the Universidade da Beira Interior, Luís Carvalho and Fernando Geraldes, for the patient support with the virtual machines, the internal network and the VPN access that made the laboratory's infrastructure reachable from outside the lab and that, in practice, made the work described in this report possible.

I would also like to thank the colleagues and researchers I had the opportunity to work with at the laboratory, for the many discussions about the legacy code base, the refactoring strategy and the painful reality of regression testing when no two responses are quite alike.

Finally, to my family, for the constant and unconditional support that allowed me to reach this point, and to the friends and colleagues who were patient enough to listen to long monologues about differential testing, JAX-RS endpoints and YAML pipelines: thank you.

Contents

List of Figures

List of Code Listings

Acronyms

API	Application Programming Interface
CD	Continuous Delivery
CDI	*Contexts and Dependency Injection*
CI	Continuous Integration
CSV	Comma-Separated Values
DDL	Data Definition Language
EPOS	European Plate Observing System
GeoJSON	Geographic JSON
GNSS	Global Navigation Satellite System
HTTP	Hypertext Transfer Protocol
IETF	Internet Engineering Task Force
JAR	Java Archive
JAX-RS	Jakarta RESTful Web Services
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JSON-LD	JSON Linked Data
JWT	JSON Web Token
LLM	Large Language Model
OpenAPI	OpenAPI Specification
POJO	Plain Old Java Object
REST	Representational State Transfer
RINEX	Receiver Independent Exchange Format
SEGAL	Space and Earth Geodetic Analysis Laboratory
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UI	User Interface

URI Uniform Resource Identifier
URL Uniform Resource Locator
VM Virtual Machine
VPN Virtual Private Network
WAR Web Application Archive
XML eXtensible Markup Language
YAML *YAML Ain't Markup Language*

Chapter

1

Introduction

1.1 Context

This document describes the work developed in the context of the final-year project of the Computer Science and Engineering programme of the Universidade da Beira Interior (UBI). The project was proposed by, and carried out at, the Space and Earth Geodetic Analysis Laboratory (SEGAL) laboratory of UBI, under the supervision of Professor Paul Andrew Crocker and the co-supervision of Vasco Senra, with day-to-day infrastructure support from Luís Carvalho and Fernando Geraldes (responsible for the laboratory's virtual machines, internal network and Virtual Private Network (VPN) access).

SEGAL is the research laboratory of UBI that operates and maintains a Global Navigation Satellite System (GNSS) data and metadata service contributing to the European Plate Observing System (EPOS), a pan-European research infrastructure focused on solid Earth science [1, 2]. As part of the EPOS Thematic Core Service for GNSS data and products, SEGAL runs a Jakarta EE Representational State Transfer (REST) Application Programming Interface (API) that exposes GNSS station metadata and observation files (in the Receiver Independent Exchange Format (RINEX) format [3]) to scientists, operators and downstream services across Europe.

The API in question has been in operation, in various forms, for several years. Over that period the code base accumulated technical debt: enourmous methods, and files of huge scale, duplicated logic copy-pasted across resources, and a long tail of patches added by successive developers and contributors to handle individual edge cases. A complete refactor of the API was therefore initiated, in which large parts of the application were rewritten from scratch on top of Jakarta EE 11 and the Payara Platform [4, 5], while a small

core of well tested logic was carried over from the previous version.

A refactor of this magnitude poses a very specific risk: any seemingly minor change in a query, a response field or a validation rule can produce subtle, hard to detect regressions that only manifest when a particular client, a particular station or a particular file is requested. Until this project, validation of those changes was carried out manually, a developer would deploy the new version, run a few requests by hand, and compare the resulting payloads against the previous one. This is what we are trying to solve.

1.2 Motivation

The motivation for this project is the gap between the pace at which the new API is being developed and the rate at which a human can reasonably validate that each new version behaves exactly like the previous one on the cases that already worked. Manual testing of a REST API that returns thousands of records, in several formats (JavaScript Object Notation (JSON), eXtensible Markup Language (XML), Comma-Separated Values (CSV)) and through dozens of query combinations, is both error-prone and slow. This does not scale to the rhythm of a continuous refactor where commits land several times a day.

The laboratory wanted an *automated* mechanism to compare every new build of the API against the previous one and flag behavioral divergences early, before they reach the production deployment. On the other hand, the laboratory also wanted to bring this mechanism inside a real Continuous Integration (CI)/Continuous Delivery (CD) pipeline running on infrastructure that the laboratory itself controls: a free GitLab instance, paired with one or more GitLab Runners hosted inside the laboratory's network.

The choice of hosting the GitLab Runners was a necessity since the free tier only allows 400 compute minutes, and due to the nature of app, and it's boot times, a full test suit usually takes up to 5 minutes per test, and being self-hosted allows us to connect to the real database infrastructure.

Beyond the immediate value delivered to the EPOS API, designing and integrating this framework was a significant personal milestone. It challenged me to collaborate with a team on a live, rolling codebase—an experience that closely mirrors real-world software engineering. Through this process, I gained practical, hands-on experience with modern Jakarta EE features, CI/CD pipelines, self-hosted infrastructure, and regression testing against a moving baseline, equipping me with industry-relevant skills that extend far beyond the scope of this project.

1.3 Objectives

In line with the project proposal, the work pursues four objectives: to design and implement an automated regression testing framework for the Jakarta EE REST API on the Payara Platform, with multi-format response comparison (JSON, XML, CSV); to integrate it into a self-managed GitLab CI/CD pipeline running on VPN-accessible runners; use of two prototype applications not only to validate the approach before applying it to the production API, but also to explore and learn about features that could be useful for the final implementation; and to produce structured per-commit comparisons at the level of Hypertext Transfer Protocol (HTTP) responses, in a form consumable by both human developers and the CI system.

The main contributions are a reusable three-stage GitLab pipeline template for Jakarta EE applications, an HTTP-level differential test harness (`HttpDiff.java`) that boots one Payara Micro per available Web Application Archive (WAR) and performs a three-way comparison between `HEAD`, the immediate parent commit `HEAD~1` and a fixed reference commit identified by the `DIFF_FIXED_BASELINE` CI/CD variable, and two prototype Jakarta EE 11 applications retained as didactic examples.

1.4 Organization of the Document

The remaining chapters of this document are organized as follows. **Chapter 2** reviews the state of the art in test frameworks, CI/CD systems, and differential testing. **Chapter 3** describes the concrete technology stack adopted during the project. **Chapter 4** explains the iterative methodology (two prototypes followed by the production application) and the overall architecture of the testing framework. **Chapter 5** applies the framework to the production API (`EPOS V4`) and walks through the pipeline, the differential harness and the reports it produces. **Chapter 6** summarizes the results, reflects on what was learned and proposes directions for future work.

1.5 Repository Access

The production API (`GNSs V4`) is hosted in a private repository restricted to laboratory members [6]. The two prototype applications developed during this work are publicly available in the author's personal namespace [7, 8].

Each repository contains a `README.md` with build instructions and (where applicable) the `.gitlab-ci.yml` configuration files discussed in Chapters 4 and 5.

Chapter

2

State of the Art

2.1 Introduction

This chapter establishes the technical background required to automatically validate a Jakarta EE REST API, build after build, against a moving reference. It explores the main families of Java testing frameworks—from standard unit tests to in-container and HTTP-level approaches—and the CI/CD orchestration tools that automate them. Because the SEGAL laboratory utilizes GitLab, special focus is given to its pipeline architecture, which directly constrained the design of this project. Finally, this chapter reviews the differential and regression testing concepts that served as the foundation for the test harness developed in Chapter 5.

2.2 Test Frameworks for the Java Ecosystem

The Java testing landscape spans multiple layers, which modern projects typically combine to ensure comprehensive coverage. At the foundational level is unit testing, where **JUnit 5** [9] serves as the de facto standard. We selected it for this project due to its first-class Maven support and its ability to generate native JUnit XML reports, which integrate seamlessly with GitLab. While **TestNG** [10] remains a viable alternative, it was deemed a legacy option for this context.

Moving up to the integration layer, Maven conventionally handles tests during its `verify` phase by executing `*IT.java` classes. To test within a container context, we evaluated **Arquillian** [11] during the first prototype. Its *micro-deployment* pattern successfully validated our initial CI/CD pipeline setup. However, Arquillian was ultimately rejected for the production API:

the application is simply too large for efficient per-test redeployments, and our methodology requires side-by-side execution rather than isolated deployments.

2.2.1 Differential Testing

The most relevant testing paradigm for this project is *differential testing*, which compares a candidate's response directly against a reference implementation rather than relying on statically defined expected outputs. A notable industry example of this concept is **Diffy** [12], which dynamically routes requests to multiple service versions to surface discrepancies side-by-side. This comparative approach addresses the exact challenge faced by the SEGAL laboratory during the refactoring of its API.

We also considered *snapshot testing*, a technique popularized by tools like Jest. However, our test infrastructure utilizes a fixed database populated with large-scale synthetic data. Freezing such massive API responses into static fixture files is highly impractical and would quickly bloat the repository. Therefore, we adopted a dynamic differential approach: rather than relying on static files or a proxy service, our harness uses the parent commit of the build under test (alongside an optional long-term reference) as the baseline. This strategy ensures comparisons remain accurate across long-running refactor branches while completely eliminating the burden of manual fixture maintenance.

2.3 Continuous Integration and Continuous Delivery

2.3.1 Foundations

CI/CD is the practice of frequently integrating code into a shared repository while keeping the codebase in a consistently deployable state through full automation. A standard deployment pipeline is structured into sequential stages typically build, test, and deploy with strict gating. A stage only executes if the previous one succeeds, ensuring that any failure immediately halts the process and alerts the development team.

The pipeline serves as the single source of truth for software quality. To prevent configuration drift between environments, a mature setup dictates that the exact artifact generated during the build stage must flow unchanged through all subsequent testing and deployment phases. This project strictly adheres to this principle: the WAR file is compiled only once during the initial

pipeline stage and is passed directly to all subsequent testing phases without modification.

2.3.2 Source Code Management with Git

Before evaluating the orchestration platforms that drive continuous integration, it is necessary to establish the underlying technology that manages the source code itself: **Git** [13]. Git is a distributed version control system where every developer maintains a complete, local copy of the repository's history. It tracks changes as a series of immutable snapshots (commits), allowing developers to work on isolated features through branching without disrupting the main codebase.

In the context of this project, Git's graph-based history is a structural requirement rather than just a collaboration tool. Because each commit strictly references its predecessor, the differential testing harness described in Section 2.2.1 can reliably identify and dynamically pull the *parent commit* to serve as the baseline for its side-by-side comparisons.

While Git provides the core version control mechanics, it is fundamentally a decentralized, command-line tool. To facilitate team collaboration at scale, organizations rely on centralized Source Code Management (SCM) platforms to host their Git repositories. Platforms such as GitHub and GitLab wrap the underlying Git protocol with web interfaces, access controls, code review mechanics (Merge Requests), and crucially for this work integrated CI/CD engines.

2.3.3 Platforms

Three major platforms dominate the modern CI/CD landscape. Evaluating them clarifies the architectural choices made for this project.

Jenkins [14] is a highly flexible, self-hosted automation server. While I have significant personal experience deploying and managing my own Jenkins infrastructure and its flexibility might have been ideal for our custom testing requirements it was ultimately rejected. Introducing a standalone CI server would impose an unnecessary maintenance burden on the SEGAL laboratory, which strongly prefers to standardize on Docker-based runners integrated with their existing tools.

GitHub Actions [15] is a popular, *YAML Ain't Markup Language* (YAML)-based CI/CD service. Although it fully supports self-hosted runners operating securely inside private networks, it was bypassed for a straightforward operational reason: the laboratory's source code management is already established on GitLab.

GitLab CI/CD [16] provides pipeline automation seamlessly integrated into the GitLab ecosystem. Pipelines are configured via a single `.gitlab-ci.yml` file, and jobs are executed by separate runner processes [17]. The SEGAL laboratory already operates a self-managed GitLab instance [18], along with a dedicated, Docker-based internal runner for the EPOS API. Because this runner is already inside the laboratory's network and can directly reach the production GNSS database without exposing it to the internet, adopting GitLab CI/CD was not an abstract preference, but the most practical choice to avoid operational friction.

2.4 Conclusion

The state of the practice in Java API testing provides a rich catalogue of tools across all layers of the stack, alongside mature CI/CD platforms to orchestrate them. From this catalogue, the project adopts JUnit 5 and the native `java.net.http.HttpClient` to build the custom test harness. A pure Java implementation was deliberately chosen over external testing tools because the existing codebase is already written in Java, ensuring the development team is working within a familiar ecosystem. Furthermore, this approach allows the harness to be executed via standard, well-documented Maven workflows, integrating seamlessly into the existing build lifecycle.

For orchestration, GitLab CI/CD was selected, a choice directly mandated by the SEGAL laboratory's established infrastructure. Conceptually, the bespoke harness described in Chapter 5 draws heavily on the differential testing principles embodied by tools like Diffy. Having established this theoretical and tooling foundation, the next chapter details the concrete technological stack upon which the implementation rests.

Chapter

3

Technologies and Tools Used

3.1 Introduction

This chapter details the specific technologies and frameworks utilized to build the solutions presented in this document. The selected stack is the result of pragmatic engineering, balancing the established architecture of the production API with modern technical requirements. Rather than enumerating these tools in isolation, the chapter examines the architecture layer by layer. It begins with API contracts and serialization formats, transitions through the Java and Jakarta EE application ecosystem and its relational databases, and finally covers the underlying servers and network tooling that support the deployment.

3.1.1 Explicit Data Serialization

Unlike typical Jakarta REST applications that rely on automatic content negotiation via `Accept` headers and transparent message body writers, the EPOS API handles serialization explicitly. Because endpoint inputs consist solely of path and query parameters rather than incoming payload bodies, the application bypasses automatic input deserialization entirely.

For outputs, the formatting logic is driven strictly by the Uniform Resource Locator (URL) path parameter. A dedicated parser component evaluates this parameter and routes the data to the appropriate conversion method. Whether the requested format is standard JSON, its JSON Linked Data (JSON-LD) or Geographic JSON (GeoJSON) variants, XML, or CSV, the parser explicitly converts the domain objects into a formatted text string. The application then manually assigns the corresponding HTTP `MediaType`

before constructing the final response.

This explicit architectural choice simplifies the validation strategy for the differential testing harness. Because all serialization regardless of the format ultimately produces a finalized string payload before leaving the Java method, the test harness can treat the resulting HTTP response body as an opaque textual blob. It simply verifies the candidate's output byte-for-byte against the parent commit's baseline, ensuring absolute formatting consistency without needing format-specific parsing logic in the test suite.

3.1.2 OpenAPI and Swagger UI

The contract of the production API is defined in an OpenAPI Specification (OpenAPI) 3.1.0 [19] document (`openapi.json`) packaged within the WAR. This document powers a Swagger User Interface (UI) [20] instance hosted at `/swagger.html`, providing an interactive, in-browser environment to explore and test the endpoints.

This setup yields two primary benefits. First, the document serves as the definitive source of truth for downstream consumers of the API, most notably the EPOS portal. Second, it enables future automated contract testing: a planned evolution of the test framework, discussed in Chapter 6, involves dynamically deriving HTTP differential test cases directly from this OpenAPI specification, rather than maintaining them as separate, static JSON definitions.

3.2 Java and Jakarta EE

The applications developed in this project target Java SE 21 [21]. The decision to utilize Java was effectively dictated by the existing API codebase. However, modern language features are leveraged extensively: `record` types are used liberally to model immutable request and response Plain Old Java Objects (POJOs), while the new pattern-matching `switch` expressions replace several complex `if/else` chains inherited from the legacy system, resulting in more maintainable code.

Jakarta EE 11 [4] serves as the foundational platform layer, with the codebase fully migrated to the modern `jakarta.*` namespace. The production application actively relies on a focused subset of core specifications: Jakarta REST 4.0 (Jakarta RESTful Web Services (JAX-RS)) [22] is used strictly for routing endpoints, while Jakarta *Contexts and Dependency Injection* (CDI) 4.1 handles all dependency injection (deliberately eschewing the standard JAX-RS `@Context` injection). Furthermore, the application utilizes Jakarta

Persistence 3.2 (Jakarta Persistence API (JPA)) [23] for object-relational mapping and Jakarta Bean Validation 3.1 for declarative constraints. Notably, standard Jakarta JSON Binding is omitted in favour of explicit serialization via Jackson, as detailed in Section 3.1.1.

During the early stages of the project, development prototypes served as a learning sandbox to evaluate the broader capabilities of the Jakarta ecosystem. These initial iterations experimented with Eclipse MicroProfile [24] features, specifically JSON Web Token (JWT) 2.1 for authentication mechanics and Health 4.0 for application probes. While these tools provided valuable insight into the platform's offerings, they were ultimately excluded from the final application in favour of a leaner, more tailored architecture.

Finally, alternative frameworks such as Spring Boot [25] were briefly considered but rejected. The existing production codebase is already built upon Jakarta EE and deployed on an established Payara server infrastructure. Attempting a parallel migration to a completely different ecosystem would have introduced significant risk and overhead without providing any proportional technical benefit.

3.3 SQL and the Database Layer

The persistence model of the production API is heavily relational: GNSS stations, networks, agencies, countries, file types, observation summaries, and embargo windows are all inextricably linked through foreign keys. Because users require highly specific data filtering, most endpoints must execute complex, conditional joins per request.

While JPA is utilized for its foundational object-relational mapping, the sheer complexity of the API's query requirements makes purely object-oriented querying (such as the JPA Criteria API) unwieldy. Instead, developers actively construct dynamic, hand-made queries through conditional string concatenations directly in Java. These hand-crafted strings are then passed to the JPA `EntityManager`, which safely binds the parameters and maps the resulting data rows back into Java POJOs. Consequently, native Structured Query Language (SQL) and its structural logic remain the true driving force of the database layer.

The laboratory has standardized on **PostgreSQL** [26] for the EPOS production database. The decision to use PostgreSQL over alternatives like **MySQL** [27] was driven primarily by historical precedent within the lab, as well as its robust support for specialized index types (such as B-tree, BRIN, and GiST) that are highly effective for large scientific datasets. Furthermore, while the PostGIS extension is not actively utilized today, standardizing on

PostgreSQL provides a realistic, low-friction pathway for future spatial data integrations.

3.4 Servers, Tooling and Network

Payara Server [5] is the Jakarta EE application server adopted by the laboratory; the production API is simply deployed onto an existing Payara installation. Its companion product, **Payara Micro**, is a single, self-contained Java Archive (JAR) that launches a deployable WAR directly from the command line. Because Payara was already the established stack, adopting Payara Micro for test orchestration was the immediate and obvious choice. It serves as the engine for the HTTP-level differential harness, booting one instance per WAR on different ports to safely replay and compare requests.

The applications are built using **Apache Maven**. The project's `pom.xml` orchestrates the build lifecycle, utilizing the `maven-war-plugin` to produce the deployable `ROOT.war`, the `maven-failsafe-plugin` for integration testing (outputting JUnit XML reports), and the `maven-dependency-plugin` to stage the Payara Micro JAR so the differential harness can spawn it as a subprocess.

GitLab [16] handles both source control and CI/CD orchestration. The laboratory hosts a self-managed GitLab instance [18], while the runner operates on a separate internal Virtual Machine (VM) executing jobs inside **Docker** [28] containers. Pipeline jobs utilize the official `maven:3.9.6-eclipse-temurin-21` image to guarantee a clean, reproducible toolchain. Under the hood, **Git** [13] is used heavily in the pipeline: the `git worktree` command materializes the baseline build alongside the candidate build on the runner's filesystem without disrupting the working copy (detailed in Section 5.2).

During early development, local database instances were simulated using Docker containers. However, this was later abandoned in favor of connecting directly to the real development database. Because the GitLab runners are provisioned on the same internal network as the EPOS database, connecting to the actual infrastructure was the most obvious and reliable choice for integration testing. The production API itself remains uncontainerized, deployed traditionally by copying the WAR onto the existing Payara server.

Finally, the production database is strictly isolated within the laboratory's internal network. While the internal runners can access it directly, developers operating remotely must connect via the university's established **OpenVPN** infrastructure, managed through pfSense. This setup utilizes existing UBI cer-

tificates, relying on proven, pre-configured network infrastructure rather than introducing new VPN solutions to the laboratory.

3.5 Conclusion

The technology stack underlying this project was largely dictated by the SEGAL laboratory's existing operational environment: Jakarta EE on Payara, a strictly internal PostgreSQL database, and a self-managed GitLab CI/CD platform. Where architectural choices were required, decisions consistently prioritized simplicity and alignment with production infrastructure over introducing novel frameworks. By standardizing on native Java tools, Payara Micro, and existing Docker-based runners, the project minimizes moving parts and avoids unnecessary maintenance overhead. With this technological foundation established, the following chapter details the iterative methodology used to synthesize these tools into a functional differential testing solution.

Chapter

4

Methodology and Architecture

4.1 The Legacy API

The starting point of the work is a code base that had grown organically for several years and that nobody fully understood anymore: the previous version of the EPOS API contained Java methods enormous scale, and source files of an even larger scale, and many subtle behavioral quirks introduced as patches by successive developers in response to specific bug reports. Consequently, the laboratory could not confidently guarantee that an arbitrary change would avoid silently breaking downstream API clients. While the full rewrite to Jakarta EE 11 significantly improved the internal architecture introducing concise methods, cohesive JAX-RS resources, and a strict separation between parsing, validation, querying, and serialization it also introduced a major regression risk. At any given commit during the refactoring process, an endpoint might produce output subtly different from the production baseline. The objective of this project is to construct a testing harness that mitigates this risk, turning subtle data deviations into immediate, actionable failures within the CI pipeline.

4.2 Iterative Methodology

The work was organized into three iterations, each carried out in its own dedicated repository. This separation served two purposes. First, it kept each iteration small and focused: the first prototype is deliberately stripped down to two trivial endpoints, so that the entire complexity is in the CI configuration and not in the application; the second prototype is the opposite, focusing on application code and leaving CI aside; the production API then brings the two

strands back together. Second, the prototypes have value in their own right as didactic artifacts that future members of the laboratory can read and re-use.

4.3 Prototype Applications

The prototype phase was divided into two distinct applications, each designed to isolate and solve a specific set of problems before merging the solutions into the production API.

4.3.1 Prototype 1: Infrastructure Validation (`java-ee-app`)

The first prototype was constructed strictly to validate the CI/CD infrastructure end-to-end before modifying the production codebase. Built as a minimal Maven WAR exposing two trivial JAX-RS endpoints, its primary purpose was to prove that Arquillian integration tests could run reliably against managed Payara Server containers provisioned directly within the GitLab pipeline.

Because this testing environment proved stable, three core pipeline patterns were carried over verbatim to the production repository:

- **Runner Isolation:** Utilizing the pinned `ubi-runner` tag to ensure jobs execute on the correct internal VM with database access.
- **Build Optimization:** Implementing a global Maven repository cache to prevent redundant dependency downloads across pipeline runs.
- **Test Visibility:** Using the `reports: junit:` directive to integrate test XML reports natively into GitLab, ensuring assertion failures correctly block the pipeline and are visible in the UI.

For a comprehensive breakdown of the containerized testing environment, Arquillian configuration, and the complete CI/CD setup, please refer to Appendix A.

4.3.2 Prototype 2: The SyncSpace Application (`syncspace-jakarta-app`)

The second prototype explored application architecture rather than pipeline infrastructure. Built as a room-booking service, SyncSpace serves as a complete, standalone Jakarta EE 11 reference application designed to exercise the modern specifications discussed in Section 3.2.

While the prototype implements a complete domain model and MicroProfile JWT security, its primary contribution to this project lies in two architectural patterns that were subsequently adopted by the production API refactoring:

- **Immutable Data Structures:** Utilizing Java `record` types for all request and response POJOs to simplify serialization and state management.
- **Centralized Error Handling:** A focused hierarchy of JAX-RS `ExceptionMappers` that intercepts all system and domain faults, translating them into structured, predictable JSON errors. This uniformity is critical, as it guarantees the differential testing harness always receives a stable, diffable payload rather than an unpredictable stack trace.

The remainder of the application's features were retained in the prototype repository as a self-contained educational resource for future members in the laboratory. For a comprehensive technical breakdown of the SyncSpace architecture, including its security flow, data access, and exception-handling diagrams, please refer to Appendix B.

4.4 Architecture of the Differential Testing Framework

Figure 4.1 illustrates the testing framework's three-layered architecture: the GitLab pipeline, the harness processes, and the application instances.

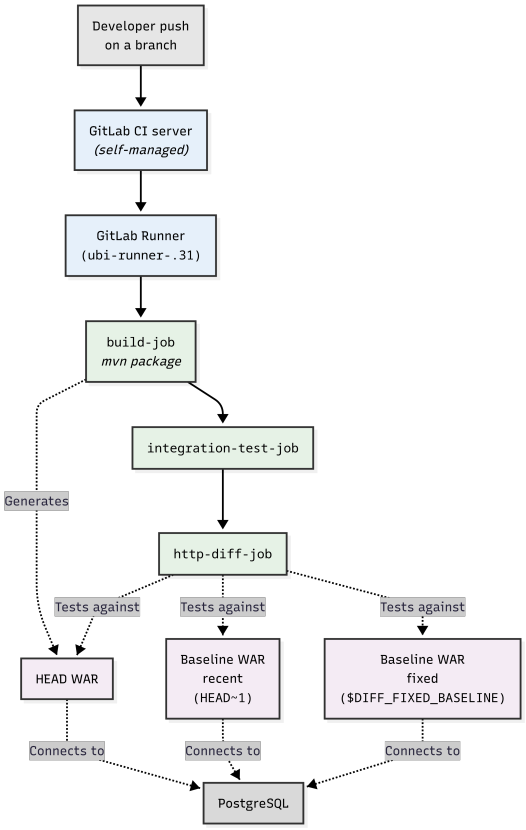

Figure 4.1: Overall architecture of the differential testing framework.

Chapter

5

Use Case: EPOS API

This chapter applies the framework of Chapter 4 to the production API, `GLASS V3`. The EPOS itself [1] is a pan-European research infrastructure in solid Earth science, organised into Thematic Core Services for seismology, geomagnetic observations, GNSS data and products [2], and so on. The SEGAL laboratory of UBI contributes to the GNSS service by operating a node that hosts station metadata and RINEX [3] observation files. `GLASS V3` serves as the public interface of that node.

Three properties of this domain shape the testing framework: the entities are relatively static (making stable baselines easy to obtain), typical client requests return extensive lists of records (requiring the comparison algorithm to scale to large diffs), and the same information is exposed across multiple serialization formats (multiplying the testing surface area without changing the underlying logic).

5.1 Architecture of `GLASS V4`

The application is a stateless Jakarta EE 11 REST service targeting Java SE 21. It is packaged as a Maven WAR (`target/ROOT.war`) and deployed onto Payara Server 6, anchoring the JAX-RS application at the root context via `@ApplicationPath("/api")`.

Its source tree is organised into a strict layered request pipeline where no state is held between calls. This stateless design is critical, as it allows the differential testing harness to boot parallel application instances simultaneously. As shown in Figure 5.1, the architecture implements a "validate-then-query" pattern, ensuring malformed requests are rejected with a `400 Bad Request` before initiating costly database round-trips. Furthermore, the core is format-

agnostic: queries return raw entities, and the requested format (JSON, XML, CSV, etc.) is applied only at final serialization.

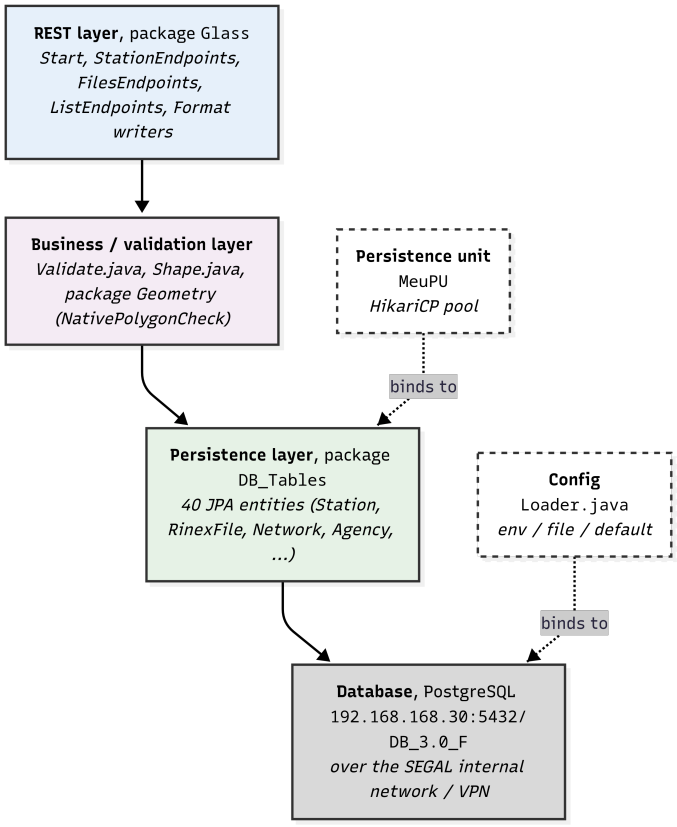

Figure 5.1: Layered architecture of EPOS V4. Solid arrows denote call/data flow; dashed arrows denote infrastructure binding.

5.1.1 Endpoints and Output Formats

The REST surface consists of four primary resource families. `Start.java` registers a `PingResource` for liveness checks (executing a trivial `SELECT 1` against the database). The core domain is handled by `StationEndpoints` (`/api/stations`) and `FilesEndpoints` (`/api/files`). Finally, `ListEndpoints` (`/api/list`) provides controlled vocabularies and lookups exclusively in JSON.

Endpoints follow a uniform grammatical structure driven heavily by path parameters (e.g., `/stations/<selector>/.../{size}/{format}`). Selectors include identifiers, locations, agencies, and coordinates. The

`combination` endpoints offer the highest flexibility, allowing an open set of query parameters validated against a strict allow-list.

For the file endpoints, responses are strictly paginated, carrying metadata headers such as `X-Total-Count`, `X-Page`, and `X-Total-Pages`. This pagination is heavily enforced due to the massive scale of the underlying dataset; attempting to query the full RINEX file corpus without limits either results in severe latency or triggers a `500 Internal Server Error` as the application exhausts memory limits.

Table 5.1 summarizes the routing structure of the API.

Endpoint Family	Example Uniform Resource Identifier (URI) Pattern
Health & Liveness	`/api/ping`
Vocabularies (Lists)	`/api/list/{entity_type}`
Station lookups	`/api/stations/stationid/{stationID}/{size}/{format}`
Location lookups	`/api/stations/location/{locationType}/{instance}/{size}/{format}`
Equipment lookups	`/api/stations/equipment/{itemType}/{instance}/{size}/{format}`
Station combinations	`/api/stations/combination/{size}/{format}?network=...`

Table 5.1: Summary of the core EPOS endpoint families and their routing structures.

It is important to acknowledge that several endpoints—specifically those targeting equipment, coordinates, and locations—maintain a deliberately non-ideal architectural implementation. As seen in the table, these endpoints map complex search criteria into deep, rigid chains of URI path segments rather than utilizing standard query parameters. While modern REST principles strongly discourage placing optional or highly variable parameters directly into the routing path, these legacy structures must be preserved exactly as-is. Active EPOS consumers, automated scripts, and federated portals are hard-coded to expect these specific paths; migrating to a standard query-string approach (e.g., `?itemType=antenna&instance=TRM`) would constitute a breaking change for the ecosystem. Furthermore, the `/combination` endpoints currently stretch the limits of the HTTP `GET` method by accepting a massive array of optional query parameters. The ideal, modern architectural

implementation for this type of complex search would be to utilize the newly proposed HTTP `QUERY` method (an active Internet Engineering Task Force (IETF) draft). The `QUERY` method provides a safe, idempotent way to retrieve data while allowing the client to send a structured JSON payload body containing all the filtering logic, bypassing URI length limits and maintaining clean routing without misusing the `POST` method.

Finally, a single `switch` statement in each resource resolves the output format matrix. Stations can be serialized to JSON, XML, CSV. Files support JSON, XML, and CSV. To optimize bandwidth, empty result sets actively short-circuit to a `200 OK` plain-text message rather than building and returning an empty structured document.

5.1.2 Persistence

Persistence is managed by Hibernate over PostgreSQL, utilising a `RESOURCE_LOCAL` unit named `MeuPU`. The connection configuration is resolved dynamically at startup by `Config.Loader`, following a strict precedence: an explicit `DATABASE_URL` environment variable overrides individual host/port variables, which override local configuration files, which finally fall back to hard-coded defaults (`192.168.168.30:5432`). This hard-coded fallback is strictly a temporary measure for the development phase and will be phased out in favor of a more robust configuration approach in the future.

This environment-variable precedence is the linchpin of the differential harness: it enables the pipeline to point multiple historical builds at a single, shared database purely by injecting `DATABASE_URL` at runtime, requiring no artifact rebuilds. As a safeguard, the pipeline enforces `JAKARTA_SCHEMA_ACTION=none` to ensure Hibernate never executes Data Definition Language (DDL) statements against live data.

To mitigate performance bottlenecks across complex joins, the architecture heavily employs batch fetching (`hibernate.default_batch_fetch_size = 512`). This collapses potentially hundreds of thousands of lazy-loading queries into highly efficient `IN (...)` clauses.

Figure 5.2: Simplified view of the core EPOS data model.

5.2 The CI/CD Pipeline Flow

The continuous integration and deployment lifecycle is orchestrated by GitLab, but the actual compilation and testing workloads are offloaded to a self-managed internal runner (using the `ubi-runner` image). This runner is provisioned within the laboratory's infrastructure, granting it secure VPN access to the reference PostgreSQL databases.

Before detailing the specific pipeline stages, it is important to understand the execution flow between the control plane and this runner, as illustrated in Figure 5.3. The runner process operates in a continuous polling loop. Once registered, it periodically requests jobs from the GitLab API. When a developer pushes a new commit, GitLab queues the pipeline jobs; the runner claims

them, pulls the job payload, and delegates the workload to its local executor. The executor then clones the repository, downloads any necessary artifacts from previous stages, runs the defined scripts, and streams the status and output back to GitLab.

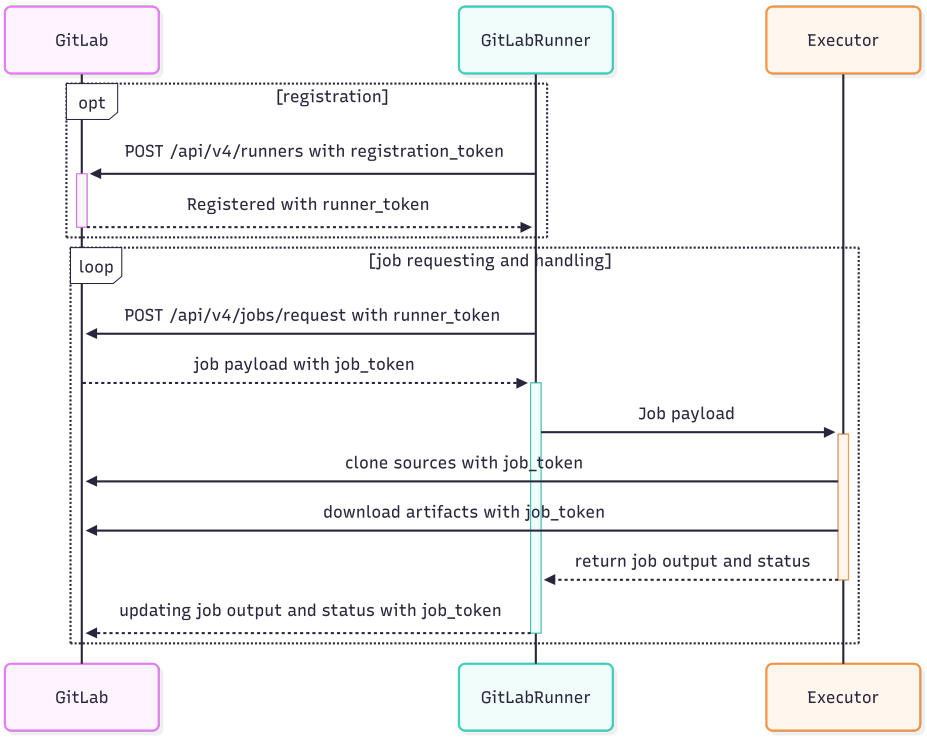

Figure 5.3: Sequence diagram of registration, polling, and execution flow.

Mapped onto this execution architecture, our specific pipeline is divided into three sequential stages: `build`, `test`, and `diff`.

5.2.1 Stage 1 & 2: Build and Unit Testing

The `build` stage initiates the process. The executor compiles the candidate `HEAD` WAR and publishes it back to GitLab as a pipeline artifact. Following this, the `test` stage downloads that artifact and executes standard JUnit 5 logic tests. Crucially, these tests are entirely isolated from the database. A deliberate design choice was made to forgo traditional data-access integration tests; instead, behavioral database correctness is strictly deferred to the differential harness.

By decoupling the unit tests from the persistence layer, the `test` stage becomes extremely fast and fiercely focused on the API's input validation boundary. The test suite, dominated by classes like `ValidateTest`, rigorously exercises the "validate-then-query" paradigm. It ensures that malformed, malicious, or illogical requests are intercepted and rejected with a `400 Bad Request` before a database connection is ever requested.

The unit testing strategy focuses on several critical vectors of input sanitization:

- **Boundary and Spatial Enforcement:** The API processes complex geographic queries. The test suite verifies that coordinate parsers enforce strict mathematical boundaries (e.g., latitude confined between -90.0 and 90.0) and logical shape constraints. For instance, tests actively assert that circles cannot have negative radii, rectangles cannot have minimum bounds exceeding maximum bounds, and polygons must contain at least three valid, non-intersecting coordinate pairs.

- **Temporal Logic:** Date parameters are validated not just for format, but for calendar reality and chronological order. The suite actively asserts that impossible dates (like February 30th) throw `IllegalArgumentException`, and that complex date ranges strictly enforce start dates that chronologically precede end dates.

- **Interdependent Query Parameters:** For the highly flexible `combination` endpoints, validation rules are contextual. The test suite proves that if a client provides an `equipmentType`, they must also provide an `equipment_instance`; similarly, location identifiers are rejected if the broader location category is missing.

- **Pagination and Resource Limits:** To protect the database from denial-of-service through massive queries, pagination limits are strictly enforced at the unit level, asserting that page sizes exceeding 1000 records or dropping below zero are immediately rejected.

By verifying these constraints in pure, database-free Java logic, the pipeline guarantees that the downstream PostgreSQL instance is insulated from bad payloads. If a commit successfully passes this stage, the subsequent differential test can safely assume that any `500 Internal Server Error` it encounters is a genuine backend logic regression, not a symptom of unhandled input.

5.2.2 Stage 3: Differential Testing (`diff`)

The final stage is the `http-diff-job`, which performs a strict three-way HTTP comparison. It relies heavily on the executor's ability to pull sources and artifacts, evaluating the compiled `HEAD` commit against two historical baselines:

- **Recent Baseline (`HEAD~1`):** Identifies immediate regressions introduced by the developer's current push.

- **Fixed Baseline (`DIFF_FIXED_BASELINE`):** A pinned, stable anchor commit that tracks cumulative drift over the lifecycle of a long-running refactor.

To execute this without polluting the active workspace, the job leverages `git worktree` to check out both baseline sources into isolated, temporary directories. It compiles them cleanly from scratch to avoid cross-contamination from stale artifacts. If a baseline is unavailable (for instance, if the repository is new and lacks sufficient commit history), the harness dynamically detects this and safely skips that specific sub-run.

5.3 The HTTP-Level Diff

`HttpDiff.java` is the testing oracle. Because hand-writing expected outputs for forty endpoints across five formats is unreliable, the harness uses previous, trusted Git commits as free ground-truth oracles.

As detailed in Figure 5.4, the harness boots up to three Payara Micro subprocesses simultaneously on separate ports (`8081` for last, `8082` for current, and `8083` for fixed). Crucially, all instances are injected with the same `DATABASE_URL` environment variable, ensuring they query identical fixtures.

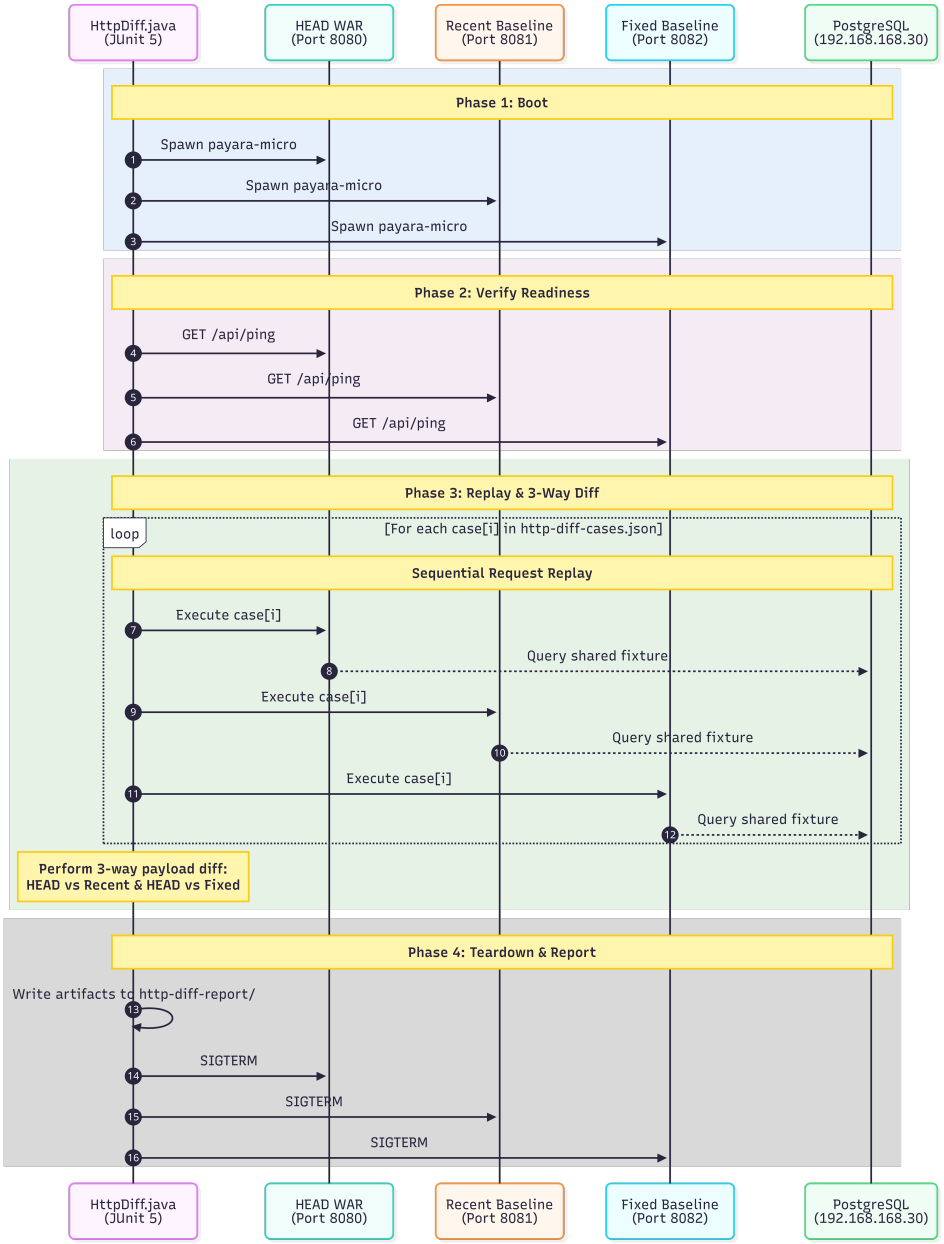

Figure 5.4: Phases of `HttpDiff.java`. The harness orchestrates boot, verification, parallel replay, and teardown.

To make the testing strategy concrete, Listing 5.1 shows a representative extract from the `http-diff-cases.json` test corpus. The file is structured as an array of JSON objects, where each object declaratively defines a request

profile.

```
[
  { "name": "ping", "method": "GET", "path": "/api/ping" },
  { "name": "stations short csv", "method": "GET", "path":
     "/api/stations/station/short/csv", "timeoutSeconds":
     120 },
  { "name": "station by id short json", "method": "GET", "
     path": "/api/stations/stationid/CASC00PRT/short/json"
     },
  { "name": "NEG unknown query param", "method": "GET", "
     path": "/api/stations/station/short/json?bogusparam=1"
     }
]
```

Code Listing 5.1: Representative extract of the `http-diff-cases.json` test corpus.

This declarative format allows the framework to evaluate roughly forty specific scenarios across three distinct dimensions of the API:

- **Serialization and Volume Matrices:** Massive full-table queries (such as retrieving the entire station catalog) place high contention on the database when executed by three instances simultaneously. These cases are explicitly granted a `timeoutSeconds` boundary to prevent false failures while still guaranteeing byte-for-byte correctness across JSON, XML, CSV formats.

- **Targeted Lookups:** Precise queries (e.g., filtering by the `CASC00PRT` marker or the `EUREF` network) verify that deeper SQL joins and parameter bindings remain structurally intact after a refactor.

- **Negative Validation (`NEG`):** Cases prefixed with `NEG` intentionally inject malformed parameters, bogus output formats, or unregistered query keys. These ensure the "validate-then-query" architecture functions correctly, strictly intercepting bad requests with a `400 Bad Request` before they reach the persistence layer.

Every case in the corpus is then evaluated by the harness on a three-tier verdict model:

- **OK:** Both instances returned identical HTTP statuses and normalized bodies.

- **WARN:** Both instances responded successfully, but their outputs differ (flagging an intentional or unintentional behavioral change).

- **FAIL:** One instance encountered an execution failure (e.g., a `5xx` server error, a connection refusal, or a timeout) while the other succeeded.

Even though the harness queries the three instances sequentially rather than concurrently, massive full-table list queries inherently require significant database processing and payload serialization time. To prevent false failures on these heavy endpoints, the harness implements per-case timeout boundaries (`timeoutSeconds`). This allows complex queries to process fully without globally lowering the performance bar for faster, lightweight requests.

5.4 Reports and Findings

Upon completion of the pipeline, both the unit testing and differential testing stages generate self-contained report trees. The pipeline is explicitly configured with the `when: always` directive to guarantee these artifacts are uploaded to GitLab even if a job fails, as failing reports provide the most actionable diagnostic data for the developer.

5.4.1 Unit Testing Artifacts

The `test` stage outputs standard JUnit XML reports (e.g., `TEST-Glass.Generics.ValidateTes`). GitLab natively ingests these XML files to populate it's Merge Request UI with test execution metrics. For instance, the core `ValidateTest` suite executes 40 discrete input-validation scenarios in approximately 50 milliseconds. Because these tests are strictly isolated from the database, the resulting reports provide instantaneous, deterministic feedback on the structural integrity of the API's boundary logic before the heavier differential harness is even invoked.

5.4.2 Differential Harness Artifacts

The `http-diff-job` stage produces a custom artifact tree designed specifically for rapid triage. A real-world execution of the harness yielded the following reporting structure:

- **`summary.json`:** The high-level entry point containing pairwise verdict tallies and performance deltas. As shown in Listing 5.2, it provides an immediate health check. In this run, the harness executed 38 test cases, yielding 37 perfect matches (`ok`) and one behavioral deviation (`warn`).

- **`report.csv`:** A row-by-row matrix detailing the HTTP status codes and wall-clock latency for every endpoint across all three Payara instances. This allows developers to track performance degradation at a glance (e.g., observing if a refactored query suddenly increased response times on the `/api/stations/location` endpoint).
- **`details.txt`:** The diagnostic core of the harness. For any case flagged as a `WARN` or `FAIL`, this file isolates the exact payload deviation. Listing 5.3 demonstrates the framework's value perfectly: it caught a subtle regression where a refactored query inadvertently changed the JSON serialization of the `location_type` field from "Country" (capitalized) to "country" (lowercase).
- **`payara-*.log`:** The raw standard-out logs for the three isolated Payara Micro processes (`current`, `last`, and `fixed`). These are essential for deep debugging when a `500 Internal Server Error` occurs, providing the full Java stack trace for the failed instance.

```
{
  "totalCases": 38,
  "fixedEnabled": true,
  "currentVsLast": {
    "ok": 37,
    "warn": 1,
    "fail": 0
  },
  "currentVsFixed": {
    "ok": 37,
    "warn": 1,
    "fail": 0
  }
}
```

Code Listing 5.2: Extract of `summary.json` showing a run with 38 total cases and 1 regression warning.

```
================================================
CASE: stations by location country json
PATH: /api/stations/location/country/Portugal/short/json
STATUS: Current=200, Last=200
--- DEVIATION (Current vs Last) ---
Current payload:
[{"station_id":"ALC000PRT","location":{"location_type":"
   country","instance":"Portugal"}}]

Last payload:
[{"station_id":"ALC000PRT","location":{"location_type":"
   Country","instance":"Portugal"}}]
================================================
```

Code Listing 5.3: Extract of `details.txt` showing the exact JSON key capitalization deviation caught by the harness.

5.4.3 Operational Realities

Several operational realities shaped this reporting design. First, database coupling is load-bearing: if the `DIFF_FIXED_BASELINE` anchor predates a major PostgreSQL schema migration, it will query mismatched data structures, resulting in a wall of bogus diffs. Thus, anchors must always be manually updated to represent compatible database eras.

Second, non-determinism does not automatically equate to behavioral change. Early test runs flagged massive amounts of `WARN`s due to the unpredictable iteration order of Java `HashSet` objects when serializing JSON arrays or generating error messages. Standardizing the API codebase to rely on sorted sets and deterministic lists eliminated this cosmetic noise, yielding a highly stable, trustworthy CI signal.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

The work described in this report solves a concrete challenge for the SEGAL laboratory: safely refactoring a legacy Jakarta EE API without silently breaking existing EPOS clients. The solution is a CI/CD-driven differential testing framework that evaluates every new commit against two baselines (the immediate parent and a fixed reference) by comparing raw HTTP responses.

Several key conclusions emerged from this project:

- **Differential testing perfectly suits refactoring:** By comparing new outputs against known-good baselines, the framework avoids the brittleness of hard-coded expected values. It provided immediate, actionable feedback from the very first commit.

- **HTTP-level comparison catches integration flaws:** Testing full HTTP responses against a live database surfaces subtle regressions that isolated unit tests naturally miss. This is especially critical during a refactor, where database interactions are the most likely point of failure.

- **Self-hosted infrastructure is mandatory:** Because the tests require access to the production database, deploying a self-managed GitLab instance and runner within the laboratory's internal network was a strict data-locality requirement, not merely a preference.

- **Prototypes mitigate risk:** Building two isolated prototypes allowed for cheap experimentation with CI configurations and Jakarta EE 11 features, keeping the production codebase clean of failed experiments.

- **Automation outperforms human review:** The framework successfully caught behavioral changes hidden inside seemingly cosmetic refactoring commits regressions that manual code review would likely have missed at the laboratory's standard commit rate.

6.2 Future Work

While the framework successfully meets its primary goals, several extensions were deliberately left for future work to further mature the system:

- **AI-Assisted Automated Triage:** Enhancing developer experience by integrating a lightweight Large Language Model (LLM) analysis step at the end of the pipeline. By pushing the `summary.json` and `details.txt` artifacts to a serverless AI service (such as Cloudflare's AI REST API) via a simple HTTP call, the system could automatically generate a structured, natural-language summary of any regressions. This would accelerate triage by posting immediate, human-readable feedback directly into the merge request, without requiring the laboratory to host its own AI infrastructure.

- **Endpoint URI Mapping (`prevURL`):** Currently, the harness assumes endpoint paths remain static. A necessary feature is the ability to map a `prevURL` to a new URI. If an endpoint is renamed or versioned, the test should know to route the baseline request to the old URL and the candidate request to the new URL, ensuring the output remains identical.

- **Performance Benchmarking:** Implementing **BRIN** indexes on the `rinex_file` date columns. This should be accompanied by a performance-regression subsystem that records wall-clock execution times against the large `DB_3.0_F` fixture to empirically measure speed improvements.

- **Spatial Database Migration:** Moving the custom spatial logic from the Java `Geometry` package directly into the database using **PostGIS**. The differential framework will be crucial here to guarantee byte-for-byte equivalence during the transition.

- **Automated Test Generation:** Dynamically generating the HTTP test cases directly from the OpenAPI document shipped inside the WAR, ensuring newly added endpoints are automatically covered by the diff harness.

Overall, the framework delivered by this project solves the problem that motivated it. It has already proven its value by catching real regressions during the refactoring process, laying a solid foundation for the laboratory to evolve the API with a much higher degree of confidence. The future work identified here represents natural next steps that build upon, rather than invalidate, the current robust design.

Appendix

A

Prototype 1: Infrastructure and CI/CD Scaffolding

This appendix outlines the architecture and configuration of the first prototype developed during this project (`java-ee-app`). Unlike the second prototype (detailed in Appendix B), which focused on Jakarta EE feature exploration, this initial application was built with a strictly constrained scope: to serve as a proof-of-concept for the underlying infrastructure. Its primary objective was to validate the integration between the Payara 7 application server, Arquillian integration testing, and the GitLab CI/CD pipeline before introducing complex domain logic.

A.1 Technology Stack

The prototype establishes the baseline toolchain that was later adapted for both the SyncSpace prototype and the production EPOS API refactoring.

Concern	Technology
Platform	Jakarta EE 11 (`jakarta.jakartaee-api 11.0.0`)
Language	Java 17
App Server	Payara 7 (`7.2026.x`)
Build	Maven 3.9.6 (WAR packaging)
Database	PostgreSQL 16 (Alpine Docker image)
REST Layer	Jakarta RESTful Web Services (JAX-RS)
Testing	Arquillian 1.10 + JUnit 5 (Payara Managed container)
CI/CD	GitLab CI

Table A.1: Prototype 1 baseline technology stack.

A.2 Architecture and Endpoints

To verify basic deployment functionality and content negotiation, the application exposes a headless REST API utilizing two independent JAX-RS `Application` subclasses. This design deliberately separates a legacy plain-text route from a versioned JSON route:

- **`HelloApplication` (`/api`):** Registers a simple `HelloResource` returning a `text/plain` fixed string (`"Hello, World!"`).
- **`JsonApplication` (`/api/v2`):** Registers a `JsonResource` returning `application/json`. It serializes a minimal `User` POJO containing an auto-generated UUID, a name, and an age.

Because the scope of this prototype was restricted to infrastructure validation, cross-cutting concerns such as security, input validation, and CDI-managed services were intentionally left unimplemented.

A.3 Persistence Readiness

While the application does not actively execute database queries, the scaffolding for persistence is fully established to validate the environment setup. A `persistence.xml` file exists defining a default persistence unit, and a PostgreSQL 16 instance (`testdb`) is provisioned locally via Docker Compose alongside the Payara server. The domain model (the `User` class) remains

a plain Java object rather than a JPA `@Entity`, leaving the final persistence wiring as a future roadmap item for the architecture.

A.4 Testing Framework

The most critical deliverable of this prototype is the integration testing harness. The project utilizes Arquillian configured for a Payara Managed container, combined with JUnit 5.

The testing lifecycle is anchored by an `AbstractBaseIT` class. This base class uses ShrinkWrap to programmatically construct a micro-WAR (`test-api.war`) from the application's source packages. It handles the setup and teardown of a JAX-RS `Client` per test and injects the dynamic deployment base URL via the `@ArquillianResource` annotation. Specific integration tests, such as `JsonApplicationIT`, inherit from this base class to issue actual HTTP requests against the running container and assert successful `200 OK` JSON responses.

A.5 CI/CD Pipeline

The application's lifecycle is automated via a `.gitlab-ci.yml` pipeline executing on a self-managed runner. The pipeline successfully validated the two-stage execution model later adopted by the broader project:

1. **Build Stage:** Executes `mvn clean package -DskipTests` within a Temurin JDK Docker image, caching dependencies in the global `.m2` repository, and publishes the resulting WAR as a pipeline artifact.
2. **Test Stage:** Boots the Payara service container, waits for readiness, and executes the Failsafe integration tests via `mvn verify`. The results are captured as JUnit XML artifacts and ingested into GitLab's native test tracking interface.

By successfully executing this pipeline, the prototype confirmed that the laboratory's self-managed runners could reliably compile Jakarta EE applications, orchestrate containerized application servers, and execute robust integration tests over HTTP.

Appendix

B

Prototype 2: SyncSpace Architecture Details

This appendix provides a comprehensive technical overview of **SyncSpace**, the second prototype developed to evaluate the Jakarta EE 11 ecosystem. SyncSpace is a backend REST API designed for coworking and meeting-room reservations. Users authenticate, browse available rooms, and reserve them either by selecting a specific room or allowing the system to auto-assign a suitable one. The application is packaged as a WAR and deployed onto a Payara 7 application server, backed by a PostgreSQL database.

B.1 Technology Stack

The prototype leverages modern enterprise Java specifications, prioritising standard APIs over third-party frameworks where possible.

Concern	Technology
Platform	Jakarta EE 11 (`jakarta.jakartaee-api 11.0.0`)
Profile APIs	MicroProfile 7.1 (JWT auth, Config)
App Server	Payara 7 (embedded for tests)
Persistence	Jakarta Persistence (JPA) via EclipseLink
Data Access	Jakarta Data repositories (`CrudRepository`)
Database	PostgreSQL (driver 42.7.10)
Security	MicroProfile JWT (RS256 via Nimbus JOSE + JWT)
Validation	Jakarta Bean Validation (Hibernate Validator)
Test	Arquillian + JUnit 5 on embedded Payara
Test Data	Datafaker 2.5.4

Table B.1: SyncSpace technology stack.

B.2 Architecture and Domain Model

The application follows a clean, layered architecture with strict separation of concerns. HTTP requests enter through JAX-RS Resources, which deserialize and validate payloads before delegating to `@ApplicationScoped` Services. These services execute business logic and interact with Jakarta Data Repositories, which in turn manage JPA Entities. Cross-cutting concerns such as exception mapping and validation surround this core flow.

The domain model relies on three primary entities:

- **User**: Contains credentials and role assignments (`USER`, `ADMIN`).
- **Room**: Defines physical spaces with a specific `RoomType` (e.g., `HOT_DESK`, `BOARDROOM`) and a maximum capacity.
- **Booking**: Links a User and a Room over a specific `startTime` and `endTime`.

B.3 Authentication and Security

Security is inherently stateless, relying on asymmetric JWT validation. The `AuthResource` handles registration and login procedures. During registration, passwords are hashed using PBKDF2WithHmacSHA256 (10,000 itera-

tions). Upon successful login, the `TokenService` issues an RS256-signed JWT containing the user's ID, username, and role claims.

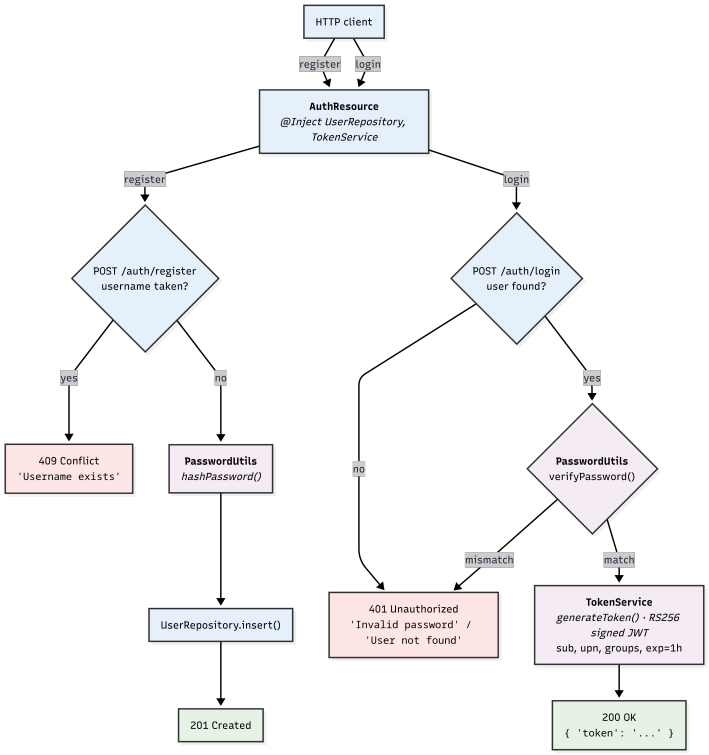

Figure B.1: Token issuance and user registration flow.

Incoming requests are intercepted by a MicroProfile JWT guard. Payara validates the signature, issuer, and expiry automatically based on a configured public key. Role-based authorization is then enforced at the endpoint level via `@RolesAllowed` annotations.

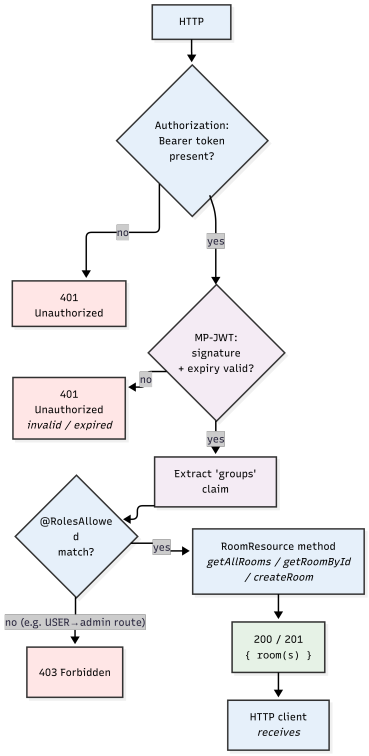

Figure B.2: Request authorization and JWT validation flow.

B.4 Booking Logic and Data Access

Data access is managed by declarative Jakarta Data interfaces (e.g., `UserRepository`, `RoomRepository`). A critical custom query exists in the `BookingRepository` to guard against double-bookings, verifying that no existing confirmed bookings for a specific room overlap with the requested time interval.

The `BookingService` executes operations within a `@Transactional` context. Notably, booking ownership is derived strictly from the JWT `sub` claim (the authenticated caller), never from client-supplied payload IDs. The service handles both specific room requests and automatic assignments, where the system queries for rooms matching the required capacity and type, streaming the results to find the first available slot.

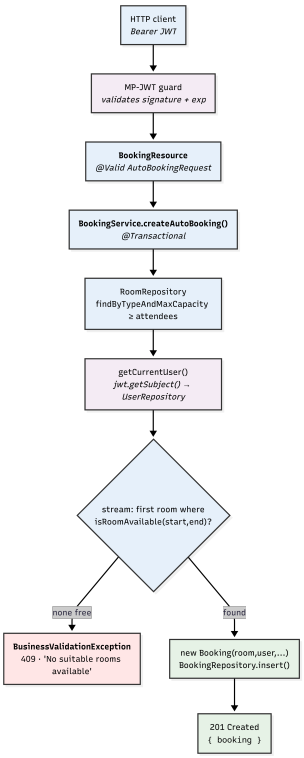

Figure B.3: Automatic booking assignment and validation flow.

B.5 API Surface

All endpoints are prefixed with `/api/v1`. The API exposes the following primary operations:

Method	Path	Auth	Purpose
POST	/auth/register	Public	Create a new user
POST	/auth/login	Public	Authenticate and obtain a JWT
POST	/bookings/auto	JWT	Auto-assign a suitable room
POST	/bookings/room/{id}	JWT	Book a specific room
GET	/rooms	USER/ADMIN	List all available rooms
POST	/rooms	ADMIN	Create a new room

Table B.2: Primary REST endpoints exposed by SyncSpace.

B.6 Validation and Centralized Error Handling

Request payloads are validated using standard Jakarta Bean Validation (`@NotNull`, `@Positive`, `@Future`). A custom class-level constraint, `@ValidDateRange`, ensures booking end times strictly follow start times, mapping violations directly to the `endTime` property node for clearer client feedback.

To ensure clients receive consistent JSON envelopes rather than raw stack traces, a four-tier JAX-RS `ExceptionMapper` hierarchy intercepts all failures.

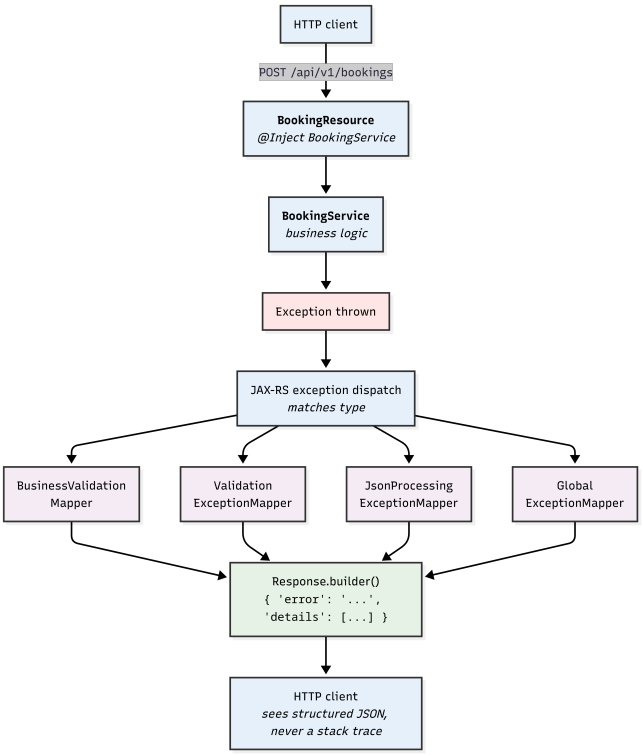

Figure B.4: The CDI and JAX-RS exception-handling flow.

The hierarchy handles specific domain exceptions (`BusinessValidationException` for capacity or availability logic), input violations (`ConstraintViolationException`), malformed payloads (`JsonProcessingException`), and falls back to a `GlobalExceptionMapper` for unexpected server faults. The global mapper securely logs the stack trace internally and returns a clean 500 response containing only a generated UUID reference.

B.7 Design Highlights

The SyncSpace prototype successfully demonstrated several architectural priorities:

- **Server-Derived Identity:** Utilizing JWT claims prevents parameter tampering and ensures users can only book resources under their own identity.
- **Stateless Execution:** Asymmetric JWT signatures eliminate the need for session state on the application server.
- **Declarative Persistence:** Jakarta Data drastically reduces boilerplate data-access code, reserving manual queries exclusively for complex overlap logic.
- **Reproducible Environments:** Integrating Datafaker with a CDI startup observer guarantees that developers and automated tests operate against a consistent, deterministic dataset on every deployment.

Appendix

C

Code Snippet: GLASS CI/CD

```
http-diff-job:
  stage: diff
  needs:
    - job: build-job
      artifacts: true
  variables:
    DATABASE_URL: "postgres://postgres:postgres@192
       .168.168.30:5432/DB_3.0_F"
    JAKARTA_SCHEMA_ACTION: "none"
    # Pinned reference commit for the current-vs-fixed
       comparison.
    FIXED_SHA: "b1809ec"
  script:
    - |
      BASELINE_SHA=$(git rev-parse --verify HEAD~1 2>/dev/
         null || echo "")
      if [ -z "$BASELINE_SHA" ]; then
        echo "HEAD has no parent commit - nothing to diff
           against. Skipping."
        mkdir -p target/http-diff-report
        echo '{"skipped": true, "reason": "no parent commit
           "}' > target/http-diff-report/summary.json
        exit 0
      fi

      build_war() {
        local ref="$1" dir="$2"
        git worktree remove --force "$dir" 2>/dev/null ||
           true
        git worktree prune
        rm -rf "$dir"
        git worktree add "$dir" "$ref"
```

```
        ( cd "$dir" && mvn -q -o package -DskipTests )
        BUILT_WAR="$dir/target/ROOT.war"
        test -f "$BUILT_WAR" || (echo "WAR not found at
            $BUILT_WAR for ref $ref" && ls -R "$dir/target"
            && exit 1)
      }

      echo "Last (HEAD~1) = $BASELINE_SHA"
      build_war "$BASELINE_SHA" /tmp/http-last-build
      export BASELINE_WAR="$BUILT_WAR"
      echo "LAST WAR     = $BASELINE_WAR"

      if [ -n "$FIXED_SHA" ]; then
        if ! git rev-parse --verify "${FIXED_SHA}^{commit}"
            >/dev/null 2>&1; then
          echo "FIXED_SHA=$FIXED_SHA is not reachable in
              this clone (GIT_DEPTH=$GIT_DEPTH). Increase
              GIT_DEPTH or pick a nearer commit." >&2
          exit 1
        fi
        echo "Fixed         = $FIXED_SHA"
        build_war "$FIXED_SHA" /tmp/http-fixed-build
        export FIXED_WAR="$BUILT_WAR"
        echo "FIXED WAR    = $FIXED_WAR"
      else
        echo "FIXED_SHA unset - running in 2-way mode (
            current-vs-last only)."
      fi

      echo "CURRENT WAR = $CI_PROJECT_DIR/target/ROOT.war"
      mvn test -Dtest=HttpDiff -DfailIfNoTests=false
  artifacts:
    when: always
    paths:
      - target/http-diff-report/
      - target/surefire-reports/
    reports:
      junit:
        - "target/surefire-reports/TEST-*.xml"
    expire_in: 4 weeks
```

Code Listing C.1: Condensed view of the production `.gitlab-ci.yml`, three stages and three jobs.

Bibliography

[1] EPOS ERIC. European Plate Observing System — EPOS, 2024. [Online] https://www.epos-eu.org/. Accessed June 2026.

[2] EPOS GNSS Data and Products Thematic Core Service. EPOS GNSS Data Gateway, 2024. [Online] https://gnss-epos.eu/. Accessed June 2026.

[3] Werner Gurtner and Lou Estey. RINEX — The Receiver Independent Exchange Format, Version 3.05, 2020. International GNSS Service (IGS), RINEX Working Group.

[4] Eclipse Foundation. Jakarta EE 11 Platform Specification, 2024. [Online] https://jakarta.ee/specifications/platform/11/. Accessed June 2026.

[5] Payara Services Ltd. Payara Platform Community Documentation, 2024. [Online] https://docs.payara.fish/. Accessed June 2026.

[6] SEGAL Laboratory. GNSs V4 Production API Repository. https://gitlab.com/segalubi/api_v4.0, 2026. Access restricted to laboratory members.

[7] Eduardo Abrantes. Java EE Prototype Application Repository. https://gitlab.com/PatalJunior/java-ee-app, 2026.

[8] Eduardo Abrantes. Syncspace Jakarta Prototype Application Repository. https://gitlab.com/PatalJunior/syncspace-jakarta-app, 2026.

[9] JUnit Team. JUnit 5 User Guide, 2024. [Online] https://junit.org/junit5/docs/current/user-guide/. Accessed June 2026.

[10] Cédric Beust. TestNG documentation, 2024. [Online] https://testng.org/. Accessed June 2026.

[11] Red Hat, Inc. Arquillian Container Testing Framework, 2024. [Online] https://arquillian.org/. Accessed June 2026.

[12] Twitter Engineering. Diffy — Automatically catch bugs in service-oriented architectures, 2015. [Online] https://blog.twitter.com/engineering/en_us/a/2015/diffy-testing-services-without-writing-tests. Accessed June 2026.

[13] Git Development Community. Git: Fast Version Control System. https://git-scm.com/, 2005. Original author: Linus Torvalds.

[14] Jenkins Project. Jenkins — The leading open source automation server, 2024. [Online] https://www.jenkins.io/. Accessed June 2026.

[15] GitHub Inc. GitHub Actions Documentation, 2024. [Online] https://docs.github.com/actions. Accessed June 2026.

[16] GitLab Inc. GitLab CI/CD documentation, 2024. [Online] https://docs.gitlab.com/ci/. Accessed June 2026.

[17] GitLab Inc. GitLab Runner documentation, 2024. [Online] https://docs.gitlab.com/runner/. Accessed June 2026.

[18] GitLab Inc. GitLab Self-Managed — Install and configure, 2024. [Online] https://about.gitlab.com/install/. Accessed June 2026.

[19] OpenAPI Initiative. OpenAPI Specification, Version 3.1.0, 2021. [Online] https://spec.openapis.org/oas/v3.1.0. Accessed June 2026.

[20] SmartBear Software. Swagger UI, 2024. [Online] https://swagger.io/tools/swagger-ui/. Accessed June 2026.

[21] Oracle Corporation. *Java Platform, Standard Edition 21 API Specification*. Oracle Corporation, 2023. [Online] https://docs.oracle.com/en/java/javase/21/.

[22] Eclipse Foundation. Jakarta RESTful Web Services 4.0, 2024. [Online] https://jakarta.ee/specifications/restful-ws/4.0/. Accessed June 2026.

[23] Eclipse Foundation. Jakarta Persistence 3.2, 2024. [Online] https://jakarta.ee/specifications/persistence/3.2/. Accessed June 2026.

[24] Eclipse Foundation. Eclipse MicroProfile, 2024. [Online] https://microprofile.io/. Accessed June 2026.

[25] VMware, Inc. Spring Boot Reference Documentation, 2024. [Online] https://docs.spring.io/spring-boot/index.html. Accessed June 2026.

[26] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, 2024. [Online] https://www.postgresql.org/docs/16/. Accessed June 2026.

[27] Oracle Corporation. MySQL 8.0 Reference Manual, 2024. [Online] https://dev.mysql.com/doc/refman/8.0/en/. Accessed June 2026.

[28] Docker, Inc. Docker Documentation, 2024. [Online] https://docs.docker.com/. Accessed June 2026.